

Aseguramiento de la Calidad y Pruebas de un Sistema de Gestión Bibliotecaria

Diego Alonso Quirós Calvo, Danny Josué Obando González, Brandon Siles Mata,
Marycruz Vargas Castro

Universidad Fidélitas, Heredia, Costa Rica

dquiros90207@ufide.ac.cr

dobando50480@ufide.ac.cr

bsiles30913@ufide.ac.cr

mvargas30767@ufide.ac.cr

Resumen— Este proyecto aborda el aseguramiento de la calidad y las pruebas de software aplicadas a un sistema de gestión bibliotecaria, con el fin de garantizar su correcto funcionamiento y el cumplimiento de los requerimientos funcionales y no funcionales definidos.

Abstract— Abstract— In this deliverable, the test plan for the Library Management System was executed and the results of 10 functional test cases were documented. The execution achieved a 60% pass rate (6 PASS, 4 FAIL) and defects related to input validation, loans, fines and reports were reported. Based on these findings, automation candidates are identified to support regression and improve product stability.

Palabras clave— Aseguramiento de calidad, pruebas del software, Pruebas funcionales, Pruebas no funcionales, Sistema de gestión de biblioteca

Key words— Quality assurance, software testing, functional testing, non-functional testing, library management system.

I. INTRODUCCIÓN

En la actualidad, los sistemas de gestión bibliotecaria son fundamentales en la administración de recursos relacionados con las bibliotecas, como usuarios, préstamos y devoluciones. Dado que estos sistemas procesan información vital para cada biblioteca, es importante garantizar su buen funcionamiento mediante procesos formales de aseguramiento de calidad del software.

La ausencia de controles de calidad en estos sistemas puede provocar registros erróneos en los datos o en el inventario, así como fallos en los procesos de préstamos y devoluciones, lo cual afecta tanto a los usuarios finales como al personal administrativo que gestiona el sistema.

Por estas razones, este proyecto aplica técnicas de aseguramiento de calidad del software al sistema de gestión bibliotecaria, validando el cumplimiento de los requerimientos funcionales y no funcionales, identificando defectos y contribuyendo a la mejora continua del software.

II. JUSTIFICACIÓN

El desarrollo del sistema de gestión bibliotecaria requiere altos estándares de calidad, dado que se maneja información sensible y crucial para la biblioteca y sus usuarios. Un fallo en

el sistema puede generar pérdida de datos, inconsistencias en los registros o interrupciones en los servicios ofrecidos, lo cual puede deteriorar la confianza de los usuarios y afectar la captación de nuevos clientes.

Según Yáñez (2023), un producto de calidad que ofrece rendimiento óptimo, facilidad de uso y cumple con las expectativas del usuario genera clientes satisfechos que confían en la empresa y recomiendan sus productos [1]. Esta afirmación refuerza la importancia de implementar procesos formales de aseguramiento de la calidad en el desarrollo de software.

En este contexto, el proyecto se justifica al aplicar técnicas de prueba que permitan detectar defectos, reducir riesgos operativos y garantizar que el sistema cumpla con los estándares de funcionalidad, seguridad y eficiencia requeridos.

III. ANTECEDENTES DEL PROBLEMA DE INVESTIGACIÓN

La gestión de bibliotecas se realizaba históricamente de manera manual, con procesos susceptibles a un alto grado de error humano, específicamente en la actualización de inventarios y en los registros de préstamos de libros. Con el avance de la era digital, surgieron los primeros sistemas de gestión bibliotecaria; sin embargo, muchos presentaban deficiencias en la calidad del software.

En el contexto actual, un software que no garantice la integridad de la información puede comprometer la operatividad de la institución y la confianza del usuario final. Esta investigación se centra en determinar cómo las pruebas de aseguramiento de calidad pueden mitigar los riesgos de pérdida de datos y mejorar la experiencia del usuario.

IV. OBJETIVO GENERAL Y ESPECÍFICOS

El objetivo general del proyecto es el siguiente:

Garantizar que el software funcione correctamente y sin errores que comprometan la estabilidad del sistema, mediante la aplicación de técnicas de aseguramiento de la calidad y pruebas de software.

Los objetivos específicos son:

1. Verificar que el sistema cumpla con los requerimientos funcionales y no funcionales definidos.
2. Identificar y documentar defectos del sistema según criterios de severidad establecidos.
3. Validar los procesos críticos del sistema, incluyendo reservas, préstamos y devoluciones.

V. MARCO METODOLÓGICO

El marco metodológico representa la sección del proyecto en la cual se describen las estrategias y métodos con los que se llevará a cabo la investigación.

Según Giani (2025), "El marco metodológico es un apartado de los textos académicos en el que se detallan los métodos de análisis, las técnicas y los instrumentos empleados para realizar una investigación." [2]

En este contexto, el marco metodológico permite seleccionar un tipo de investigación adecuado y definir la población y muestra sobre la cual se aplicarán las pruebas.

A. Tipo de Investigación

Para este proyecto se seleccionó un enfoque de investigación cuantitativa, orientado a la recolección de datos mediante formularios de evaluación, con el fin de analizar estadísticamente las respuestas e identificar problemas o áreas de mejora en el software.

Según Gayubas (2025), "A partir de los datos obtenidos, conduce análisis matemáticos o estadísticos para describir el comportamiento de variables, identificar tendencias o patrones, testear hipótesis y, a veces, predecir fenómenos." [3]

Con este enfoque es posible analizar las respuestas del formulario y detectar errores y defectos en el sistema de manera estructurada.

B. Población y Muestra

La población de estudio está compuesta por los módulos funcionales del sistema bibliotecario, incluyendo la gestión de usuarios, registro y actualización de libros, control de préstamos y devoluciones, consultas y generación de reportes, entre otras funcionalidades.

Asimismo, se considera como parte de la muestra a los usuarios finales del sistema, como el personal administrativo y los usuarios de la biblioteca.

C. Aplicación de Pruebas de Software

Para garantizar la calidad del sistema, se aplicarán los siguientes tipos de pruebas:

Pruebas funcionales: Se ejecutarán casos de prueba diseñados para validar cada requerimiento funcional y verificar su correcto funcionamiento. Se verificará el registro de usuarios, préstamos, devoluciones, reservas y generación de reportes.

Pruebas de integración: Se evaluará la interacción entre los distintos módulos del sistema, asegurando que la comunicación entre la aplicación y la base de datos funcione correctamente.

Pruebas de regresión: Cada vez que se realice una

corrección o mejora, se repetirán los casos de prueba anteriores para asegurar que no se introduzcan nuevos errores en funcionalidades previamente validadas.

Pruebas de usabilidad: Se aplicarán encuestas a los usuarios para evaluar la facilidad de uso, la claridad de la interfaz y la experiencia general con el sistema.

Pruebas de rendimiento: Se medirán los tiempos de respuesta del sistema en operaciones como búsqueda y registro de préstamos, con el fin de validar su eficiencia bajo condiciones normales de carga.

D. Métricas de Evaluación

Para medir la calidad del software se utilizarán las siguientes métricas:

- Porcentaje de casos de prueba aprobados sobre el total ejecutado.
- Número total de defectos identificados durante la ejecución.
- Clasificación de defectos por severidad: crítico, mayor o menor.
- Tiempo promedio de respuesta del sistema.
- Nivel de satisfacción del usuario según los resultados de las encuestas.
-

E. Criterios de Aceptación

El sistema se considerará apto si cumple con los siguientes criterios:

- Al menos el 90% de los casos de prueba deben ser exitosos.
- No deben existir defectos de severidad crítica al finalizar el ciclo de pruebas.
- El tiempo de respuesta no debe superar los 2 segundos.
- El nivel de satisfacción del usuario debe ser calificado como alto en las encuestas de usabilidad.

VI. DESCRIPCIÓN DE LOS INSTRUMENTOS

Para la ejecución del aseguramiento de calidad del sistema, se utilizarán los siguientes instrumentos técnicos de recolección y documentación de resultados:

1. *Casos de prueba estructurados:* Se elaborará un conjunto de pruebas documentadas con los siguientes campos: identificador del caso, requerimiento asociado, precondiciones, pasos de ejecución, resultado esperado, prioridad, tipo de ejecución, fecha y resultado obtenido. Este instrumento permite verificar de manera objetiva si cada funcionalidad cumple con lo requerido. (Ver Anexo B)
2. *Plan de pruebas estructurado:* Se elaborará un plan de pruebas que incluye: alcance (en scope / out of scope), objetivos de calidad, roles y responsabilidades, niveles de prueba, proceso de triage de bugs y criterios de suspensión, reanudación y completitud. (Ver Anexo A)
3. *Encuestas de usabilidad:* Se aplicarán encuestas a los usuarios del sistema para evaluar la facilidad de uso, la claridad de la interfaz, la comprensión de las funcionalidades y el nivel general de satisfacción.

4. *Ambiente controlado de pruebas:* Las pruebas se ejecutarán en un entorno compuesto por la aplicación Java, la base de datos MySQL y la conexión mediante JDBC.
5. *Matriz de trazabilidad de requerimientos:* Se elaborará una matriz que vincula cada requerimiento funcional y no funcional con sus respectivos casos de prueba y los resultados obtenidos, permitiendo evidenciar la cobertura alcanzada. Esta matriz se adjunta en el Anexo C junto al resumen del plan de pruebas.

RF	DESCRIPCIÓN
01 Gestión de usuarios	El sistema deberá permitir el registro de nuevos usuarios mediante un formulario que solo valide datos obligatorios como el nombre, identificación y correo. El registro será exitoso únicamente si no existe previamente un usuario con el mismo ID.
02 Autenticación	El sistema permitirá a los usuarios autenticarse mediante credenciales como el usuario o contraseña, validando la información con la base de datos y restringiendo el acceso en caso de datos incorrectos
03 Gestión de inventario	El sistema deberá permitir al administrador agregar, editar y eliminar libros del inventario, actualizando la disponibilidad en la base de datos.
04 Búsqueda de libros	El sistema deberá permitir a los usuarios realizar búsquedas por título, autor o categoría, mostrando resultados en un tiempo menor a 2 segundos.
05 Préstamo de libros	El sistema deberá permitir a un usuario autenticado solicitar el préstamo de un libro disponible, registrando la fecha del préstamo y la fecha límite de devolución.
06 Devolución de libros	El sistema deberá permitir registrar la devolución de un libro, actualizando su estado a disponible y registrando la fecha de entrega.
07 Reserva de libros	El sistema deberá permitir a los usuarios reservar libros que se encuentren actualmente prestados, bloqueando su disponibilidad para otros usuarios hasta que se liberen.
08 Renovación de préstamo	El sistema deberá permitir renovar un préstamo siempre que el libro no tenga reservas activas por parte de otro usuario.

09 Generación de reportes	El sistema deberá permitir al administrador generar reportes sobre préstamos activos, historial de usuarios e inventario disponible.
10 Historial de operaciones	El sistema deberá almacenar un historial de préstamos y devoluciones asociado a cada usuario, permitiendo su consulta posterior.

VIII. REQUERIMIENTOS NO FUNCIONALES

RNF	DESCRIPCIÓN
01 Integridad de los datos	El sistema deberá garantizar que no existan registros duplicados de usuarios mediante restricciones de clave primaria y validaciones en la base de datos.
02 Usabilidad	El sistema deberá permitir que un usuario nuevo complete un préstamo en un máximo de 3 minutos sin asistencia externa. Además, la interfaz deberá mantener consistencia visual y mensajes claros.
03 Disponibilidad	El sistema deberá garantizar una disponibilidad mínima del 95% mensual, excluyendo mantenimientos programados.
04 Seguridad	El sistema deberá implementar autenticación obligatoria para acceder a funcionalidades internas y control de roles que restrinja acciones administrativas únicamente a usuarios autorizados.
05 Eficiencia	El tiempo de respuesta del sistema no deberá superar los 2 segundos en operaciones de búsqueda y registro bajo condiciones normales de carga (hasta 20 usuarios simultáneos).

IX. LENGUAJES Y TECNOLOGÍAS UTILIZADAS

Para el desarrollo y ejecución del sistema se utilizaron dos tecnologías principales bajo una arquitectura cliente-servidor, lo que permite la separación entre la lógica del negocio y el almacenamiento de datos.

- *Java*: Lenguaje de programación utilizado para el desarrollo de la aplicación, elegido por su portabilidad y tipado estático, lo que facilita la detección de errores en tiempo de compilación.
- *MySQL*: Sistema de gestión de bases de datos relacionales utilizado para el almacenamiento persistente de la información de usuarios, libros y demás entidades del sistema, garantizando la integridad referencial.
- *JDBC (Java Database Connectivity)*: API de Java que permite la conexión y ejecución de sentencias SQL entre la aplicación y el servidor de bases de datos MySQL.
- *Arquitectura cliente-servidor*: Estructura que permite a múltiples terminales interactuar con un repositorio centralizado de datos, optimizando la gestión de los recursos de la biblioteca.

X. HERRAMIENTA PARA PRUEBAS NO FUNCIONALES SELECCIONADA

Para la evaluación de los requerimientos no funcionales relacionados con el rendimiento del sistema, se seleccionó Apache JMeter como herramienta principal para la ejecución de pruebas no funcionales.

Apache JMeter es una herramienta ampliamente utilizada para la ejecución de pruebas de rendimiento, carga y estrés en aplicaciones cliente-servidor.

Según Shethi (2024), "Apache JMeter es una plataforma de código abierto para evaluar el rendimiento de sitios web y aplicaciones desde 1999. JMeter se creó inicialmente como una herramienta de pruebas de carga para el servidor HTTP Apache. Sin embargo, durante los últimos 20 años, se ha expandido hasta convertirse en una potente plataforma de pruebas de rendimiento utilizada por ingenieros, desarrolladores y equipos DevOps de todo el mundo." [4]

La elección de esta herramienta se fundamenta en las siguientes razones:

- El sistema bibliotecario sigue una arquitectura cliente-servidor, la cual es compatible con el modelo de pruebas de JMeter.
- JMeter proporciona métricas detalladas como tiempo de respuesta, porcentaje de errores y rendimiento bajo múltiples usuarios concurrentes.

Escenarios de aplicación representativos incluyen la simulación de 20 usuarios realizando búsquedas simultáneas y 10 usuarios registrando préstamos al mismo tiempo, lo que permite identificar posibles cuellos de botella en consultas SQL.

Los resultados obtenidos mediante JMeter serán integrados al

informe final de calidad, documentando el comportamiento del sistema bajo carga y estableciendo si se cumple el requerimiento no funcional de eficiencia (RNF-05). En caso de detectarse incumplimientos, se generarán defectos clasificados según el proceso de triage definido en el plan de pruebas.

XI.EVIDENCIA DE LA EJECUCION DE LOS CASOS DE PRUEBA

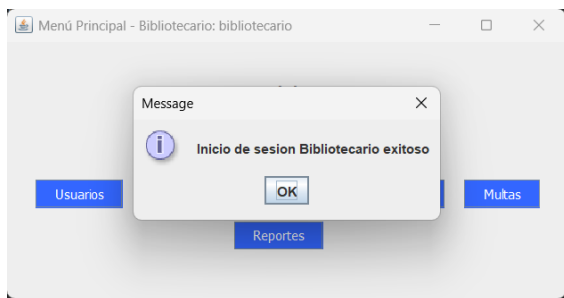
TC-01 - Iniciar sesión con credenciales válidas de Bibliotecario

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: PASS

Observaciones: El sistema autenticó correctamente al usuario y mostró el menú del bibliotecario.



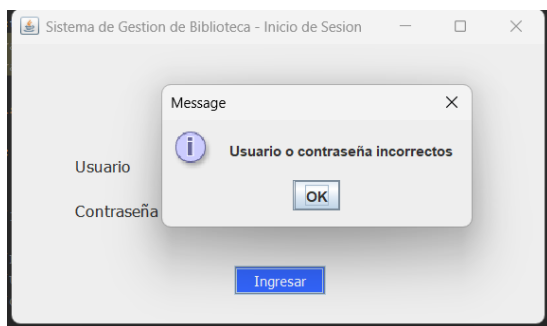
TC-02 - Bloquear acceso con credenciales inválidas

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: PASS

Observaciones: El sistema mostró mensaje de error y no permitió el acceso.



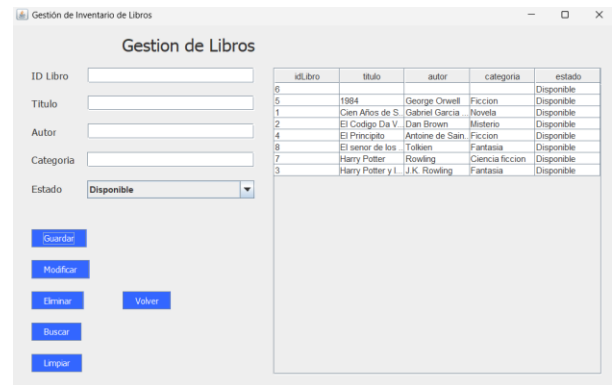
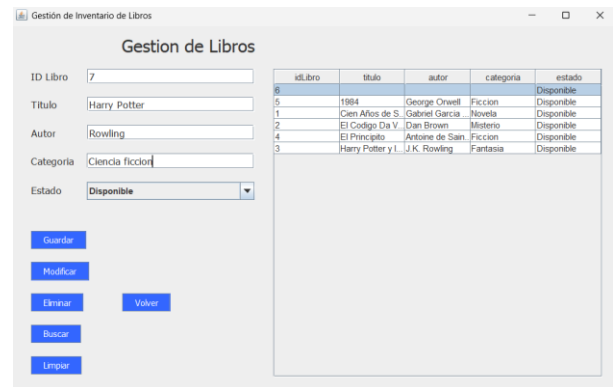
TC-03 - Registrar un libro con datos válidos

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: PASS

Observaciones: El libro se registró correctamente aunque el ID asignado fue diferente al ingresado. Ver BUG-001.



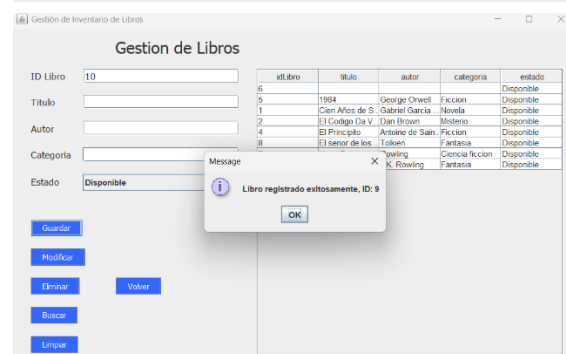
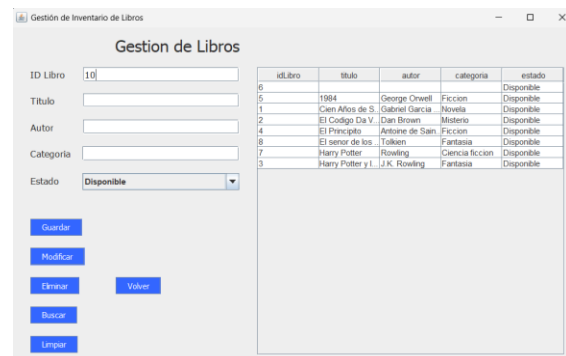
TC-04 - Validar campos obligatorios al registrar libro (título vacío)

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: FAIL

Observaciones: El sistema permitió guardar el libro sin título. Ver BUG-002.



TC-05 - Registrar un usuario Estudiante con datos válidos

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: PASS

Observaciones: El usuario se registró correctamente y aparece en el listado.

The screenshot shows the 'Gestion de Usuarios' interface. On the left, there is a form with fields for 'ID Usuario' (E003), 'Cedula' (120090207), 'Nombre' (estudiante3), 'Correo' (estudiante3@sistema.com), 'Rol' (Estudiante), and 'Estado' (Activo). Below the form are buttons for 'Guardar', 'Modificar', 'Eliminar', 'Volver', 'Buscar', and 'Limpiar'. On the right, there is a table of existing users:

idUsuario	Cedula	Nombre	Correo	Rol	Estado
B001	12345678	bibliotecario	bibliotecario	Bibliotecario	Activo
E001	87654321	estudiante	estudiante@	Estudiante	Activo
E002	11223344	estudiante2	estudiante2	Estudiante	Activo
E003	120090207	estudiante3	estudiante3	Activo	Activo

TC-06 - Registrar préstamo cuando el libro está disponible

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: FAIL

Observaciones: El préstamo no se registra bien y muestra un mensaje que el libro no está disponible

The screenshot shows the 'Gestion de Prestamos' interface. On the left, there is a form with fields for 'ID Prestamo' (100), 'ID Libro' (3), 'ID Usuario' (E002), and 'Fecha Prestamo' (2026-03-10). Below the form are buttons for 'Reservar', 'Cancelar Reserva', and 'Volver'. On the right, there is a table of existing loans:

ID Prestamo	ID Libro	ID Usuario	Fecha Prestamo	Estado
100	3	E002	2026-03-10	Activo

TC-07 - Impedir préstamo cuando el libro no tiene disponibilidad

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: PASS

Observaciones: El sistema mostró el mensaje "El libro no está Disponible, Revise si esta Prestado o Reservado" correctamente.

The screenshot shows the 'Gestion de Prestamos' interface. On the left, there is a form with fields for 'ID Prestamo' (100), 'ID Libro' (3), 'ID Usuario' (E002), and 'Fecha Prestamo' (2026-03-10). Below the form are buttons for 'Reservar', 'Cancelar Reserva', and 'Volver'. On the right, there is a table of existing loans:

ID Prestamo	ID Libro	ID Usuario	Fecha Prestamo	Estado
100	3	E002	2026-03-10	Activo

TC-08 - Crear reserva de un libro no disponible

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: PASS

Observaciones: La reserva se registró correctamente en la base de datos aunque no se mostró en la tabla de la interfaz. Ver BUG-005.

The screenshot shows the 'Gestion de Reservas' interface. On the left, there is a form with fields for 'ID Reserva' (1), 'ID Libro' (3), and 'ID Usuario' (E002). Below the form are buttons for 'Reservar', 'Cancelar Reserva', and 'Volver'. On the right, there is a table of existing reservations:

ID Reserva	ID Libro	ID Usuario	Fecha Reserva	Estado
1	3	E002		Activo

TC-09 - Consultar multa por préstamo atrasado

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: FAIL

Observaciones: El módulo de multas mostró textos de desarrollo sin reemplazar. Ver BUG-006.

TC-10 - Generar reporte de préstamos por rango de fechas

Fecha de ejecución: 2026-03-20

Ejecutado por: Diego Quiros

Resultado: FAIL

Observaciones: El sistema no encontró datos para el reporte aunque existen registros en la base de datos. Ver BUG-007.

- TC-01 y TC-02 (Autenticación): Automatizables. Flujos críticos, repetibles y con validaciones claras (acceso permitido / mensaje de error).
- TC-03 y TC-04 (Gestión de libros y validaciones): Automatizables. Permiten detectar regresiones en registro y validación de campos obligatorios (ej. título vacío).
- TC-05 (Registro de usuario): Automatizable. Caso repetible con verificación de creación en BD/listado.
- TC-06 y TC-07 (Préstamos y disponibilidad): Automatizables (post-fix en TC-06). Funcionalidad central; automatizar reglas de disponibilidad y confirmación de préstamo.
- TC-08 (Reservas): Parcialmente automatizable. Se recomienda automatizar la inserción/verificación en BD y, una vez corregida la UI, la verificación en tabla.
- TC-09 y TC-10 (Multas y reportes): Automatizables (post-fix). Requieren datos semilla controlados; validación por contenido (monto, registros en reporte).

Se automatizar 3 de los casos de prueba, de entre los cuales están: Validar login con credenciales correctas e incorrectas, y validar campos obligatorios a la hora de registrar un libro.

```
82 // Verificar que el titulo sigue vacio (no guardo)
83 WebElement campoTitulo = driver.findElement(By.id("txtTitulo"));
84 String valorTitulo = campoTitulo.getAttribute("value");
85 System.out.println("Valor del titulo: " + valorTitulo);
86
87 Assertions.assertEquals(
88     "", valorTitulo,
89     "FAIL: El sistema permitió guardar un libro sin titulo"
90 );
91 }
92
93 // =====
94 // TC-04: Validar login con credenciales incorrectas
95 // Resultado esperado: PASS - muestra mensaje de error
96 // =====
97 @Test
98 @Order(3)
99 @DisplayName("TC-04 - Validar login con credenciales incorrectas")
100 public void tc04_loginCredencialesIncorrectas() {
101     driver.get(BASE_URL + "/login");
102
103     driver.findElement(By.id("txtUsuario")).clear();
104     driver.findElement(By.id("txtUsuario")).sendKeys("usuarioMalo");
105     driver.findElement(By.id("txtContraseña")).clear();
106     driver.findElement(By.id("txtContraseña")).sendKeys("contrasenaErronea");
107
108     try { Thread.sleep(1000); } catch (InterruptedException e) {}
109     driver.findElement(By.id("btnIngresar")).click();
110     try { Thread.sleep(2000); } catch (InterruptedException e) {}
111
112     // Verificar que sigue en /login (no entró)
113     String urlActual = driver.getCurrentUrl();
114     System.out.println("URL despues del intento: " + urlActual);
115     Assertions.assertTrue(
116         urlActual.contains("login"),
117         "FAIL: El sistema dejó entrar con credenciales incorrectas"
118     );
119
120     // Verificar que muestra el mensaje de error
121     WebElement error = driver.findElement(By.id("mensajeError"));
122     System.out.println("Mensaje de error: " + error.getText());
123     Assertions.assertTrue(
```

XII. AUTOMATIZACIÓN DE CASOS DE PRUEBA

A partir de los casos de prueba ejecutados (TC-01 a TC-10), se identificaron como candidatos a automatización aquellos casos repetibles, de alta criticidad y con salidas verificables. En un sistema de escritorio Java (Swing) la automatización puede realizarse mediante pruebas unitarias/integración con JUnit sobre la capa de consultas/controladores, y pruebas de interfaz con frameworks de GUI

```

41 // =====
42 // TC-01: Iniciar sesión con credenciales válidas
43 // Resultado esperado: PASS
44 // =====
45 @Test
46 @Order(1)
47 @DisplayName("TC-01 - Iniciar sesión con credenciales válidas")
48 public void tc01_loginCredencialesValidas() {
49     hacerLogin();
50
51     String urlActual = driver.getCurrentUrl();
52     System.out.println("URL después del login: " + urlActual);
53
54     Assertions.assertTrue(
55         urlActual.contains("menu-bibliotecario"),
56         "FAIL: No redirigió al menú del bibliotecario. URL fue: " + urlActual
57     );
58 }
59
60 // =====
61 // TC-04: Validar campos obligatorios al registrar libro
62 // Resultado esperado: el sistema NO debe permitir guardar sin título
63 // =====
64 @Test
65 @Order(2)
66 @DisplayName("TC-04 - Validar campos obligatorios al registrar libro")
67 public void tc04_validarCamposLibro() {
68     hacerLogin();
69     driver.get(BASE_URL + "/libros");
70     try { Thread.sleep(1500); } catch (InterruptedException e) {}
71
72     // llenar solo autor y categoría, dejar título vacío
73     driver.findElement(By.id("txtAutor")).clear();
74     driver.findElement(By.id("txtAutor")).sendKeys("Autor Test");
75     driver.findElement(By.id("txtCategoria")).clear();
76     driver.findElement(By.id("txtCategoria")).sendKeys("Categoria Test");
77
78     driver.findElement(By.id("btnGuardar")).click();
79     try { Thread.sleep(1500); } catch (InterruptedException e) {}
80 }
81 package com.biblioteca.tests;
82
83 import io.github.bonigarcia.wdm.WebDriverManager;
84 import org.junit.jupiter.api.*;
85 import org.openqa.selenium.*;
86 import org.openqa.selenium.WebDriver;
87 import org.openqa.selenium.WebElement;
88 import org.openqa.selenium.chrome.ChromeDriver;
89 import org.openqa.selenium.chrome.ChromeOptions;
90
91 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
92 public class CasosPruebaTest {
93
94     WebDriver driver;
95     final String BASE_URL = "http://localhost:8080";
96
97     @BeforeEach
98     public void setUp() {
99         WebDriverManager.chromedriver().setup();
100         ChromeOptions options = new ChromeOptions();
101         options.addArguments("--start-maximized");
102         driver = new ChromeDriver(options);
103     }
104
105     @AfterEach
106     public void tearDown() {
107         if (driver != null) driver.quit();
108     }
109
110     // Helper para no repetir el login en cada test
111     private void hacerLogin() {
112         driver.get(BASE_URL + "/login");
113         driver.findElement(By.id("txtUsuario")).clear();
114         driver.findElement(By.id("txtUsuario")).sendKeys("bibliotecario");
115         driver.findElement(By.id("txtContraseña")).clear();
116         driver.findElement(By.id("txtContraseña")).sendKeys("biblioteca123");
117         try { Thread.sleep(1000); } catch (InterruptedException e) {}
118         driver.findElement(By.id("btnIngresar")).click();
119         try { Thread.sleep(2000); } catch (InterruptedException e) {}
120     }
121 }

```

XIII. INFORME DE CIERRE DE PRUEBAS

Este informe resume el cierre del ciclo de pruebas ejecutado sobre el Sistema de Gestión Bibliotecaria, con base en los resultados de ejecución y los defectos identificados.

- Ambiente de pruebas: Aplicación Java (Swing) conectada a MySQL mediante JDBC, ejecutada en un entorno controlado de laboratorio.
- Alcance ejecutado: Pruebas funcionales sobre autenticación, gestión de libros, registro de usuarios, préstamos, reservas, multas y reportes (TC-01 a TC-10).
- Resumen de ejecución: Fecha de ejecución principal: 2026-03-20. Total de casos ejecutados: 10.
- Métricas de resultados: PASS: 6, FAIL: 4. Porcentaje de aprobación: 60%.
- Defectos relevantes: Se registraron defectos asociados a: BUG-001, BUG-002, BUG-005, BUG-006, BUG-007 y un defecto

asociado a préstamo (TC-06).

- Evaluación de criterios de aceptación: No cumple el criterio de $\geq 90\%$ de casos exitosos (se obtuvo 60%). Existen defectos que afectan procesos críticos (préstamos y reportes).
- Recomendación de cierre: Estado del producto: NO APTO para liberación en su estado actual. Se recomienda corregir defectos de severidad alta/crítica, re-ejecutar TC-04, TC-06, TC-09 y TC-10, y repetir un ciclo de regresión completo.
- Próximos pasos: 1) Corrección de defectos y retesteo. 2) Ejecución de regresión. 3) Implementación progresiva de automatización para TC críticos. 4) Ejecución de pruebas de rendimiento (RNF-05) con JMeter y documentación de resultados.

REFERENCIAS

- [1] Yáñez, Carlos. (2023, 30 de noviembre). ¿Por qué la calidad es importante en el desarrollo de software? [Online]. Disponible en: <https://socratech.es/blog/calidad-es-importante-en-el-desarrollo-de-software/>
- [2] Giani, Carla (2025, 5 de agosto). Marco metodológico. Enciclopedia Concepto. [Online]. Disponible en: <https://concepto.de/marco-metodologico/>
- [3] Gayubas, Augusto (2025, 18 de septiembre). Investigación cuantitativa. Enciclopedia Concepto. [Online]. Disponible en: <https://concepto.de/investigacion-cuantitativa/>
- [4] Shethi, Satish (2024, 3 de septiembre). Apache Jmeter: Todo lo que necesita saber. GeekFlare. [Online]. Disponible en: <https://geekflare.com/es/apache-jmeter-guide/>

ANEXOS

Anexo A: Plan de Pruebas. Se adjunta en formato PDF con el plan de pruebas detallado.

Anexo B: Casos de Prueba. Se adjunta en formato Excel con los casos de prueba diseñados y el estado de ejecución de cada uno.

Anexo C: Resumen del Plan de Pruebas y Matriz de Trazabilidad. Se adjunta en formato Excel con el resumen ejecutivo del plan de pruebas y la matriz de trazabilidad que vincula cada requerimiento con sus casos de prueba correspondientes.

Anexo D: Se adjunta en formato PDF la plantilla de Bug Report donde se evidencian todos los bugs presentes en el sistema.

